VIKNESH.G 23MID0161 DIGITAL ASSIGNMENT-2

1.WRITE A CODE TO RETURN NO.OF PRIME NUMBERS AND LIST THEM OUT WITHOUT TRAVESING THE

COMPLETE N TERMS AND FOR THE LIST OUT THE PRIME NUMBER FOR SPECIFIED RANGE

In [16]:
```
#exection of simple sieve
print("1. Simple sieve")
print("2. Segmented Sieve")

choice = int(input("Enter your choice: "))

if choice == 1:

    n = int(input("Enter n: "))

    if n <= 2:
        print(0)
    else:
        prime = [True] * n
        prime[0] = False
        prime[1] = False
        for i in range(2, int(n**0.5)+ 1):
            if prime[i]:
                for j in range(i*i, n, j):
                    prime[j] = False

        count = 0
        print("Prime numbers are:")
        for i in range(n):
            if prime[i]:
                print(i, end=" ")
                count += 1
        print("\nNumber of prime numbers =", count)
```

```
elif choice == 2:

    low = int(input("Enter low: "))
    high = int(input("Enter high: "))

    prime = [true] * (high - low + 1)

    for i in range(2, int(high**0.5) + 1):
        start = max(i * i, ((low+ i - 1)//i) * i)
        for j in range(star,high + 1, i):
            prime[j -high] = False

    count = 0

    print("Prime numbers are:")
    for i in range(low, high + 1):
        if i > 1 and prime[i - low]:
            print(i, end=" ")
            count += 1
    print("\nNumber of prime numbers =", count)
else:
    print("Invalid Choice")
```

```
1. Simple sieve
2. Segmented Sieve
Prime numbers are:
2 3 5 6 7 10 11 13 14 15 17 18 19 21 22 23
Number of prime numbers = 16
```

logical error returns all the number xxx

In [3]:
```
#exection of segmented  sieve
print("1. Simple sieve")
print("2. Segmented Sieve")

choice = int(input("Enter your choice: "))

if choice == 1:

    n = int(input("Enter n: "))
```

```
    if n <= 2:
        print(0)
    else:
        prime = [True] * n
        prime[0] = False
        prime[1] = False
        for i in range(2, int(n ** 0.5) + 1):
            if prime[i]:
                for j in range(i * i, n, i):
                    prime[j] = False

        count = 0
        print("Prime numbers are:")
        for i in range(n):
            if prime[i]:
                print(i, end=" ")
                count += 1
        print("\nNumber of prime numbers =", count)
elif choice == 2:

    low = int(input("Enter low: "))
    high = int(input("Enter high: "))

    prime = [True] * (high - low + 1)

    for i in range(2, int(high ** 0.5) + 1):
        start = max(i * i, ((low + i - 1) // i) * i)
        for j in range(start, high + 1, i):
            prime[j - low] = False

    count = 0

    print("Prime numbers are:")
    for i in range(low, high + 1):
        if i > 1 and prime[i - low]:
            print(i, end=" ")
            count += 1
    print("\nNumber of prime numbers =", count)
else:
    print("Invalid Choice")
```

```
1. Simple sieve
2. Segmented Sieve
Prime numbers are:
83 89
Number of prime numbers = 2
```

explaination:

- simple sieve -its is used to find the prime numeber without travesing complete n number with the of removing the multiple of sqrt.of n value

- segmeneted sieve -it is used to find the no.of prime number in the short listed array only by checking with the shortied range

manual calculations attached below

2. Maximal Rectangle

In [6]:
```python
def maximalRectangle(matrix):

    if not matrix:
        return 0

    rows = len(matrix)
    cols = len(matrix[0])
    max_area = 0

    for top in range(rows):

        temp = [1] * cols

        for bottom in range(top, rows):

            for col in range(cols):
                if matrix[bottom][col] == '0':
                    temp[col] = 0

            width = 0
```

```
            for col in range(cols):
                if temp[col] == 1:
                    width += 1
                    area = width * (bottom - top + 1)
                    if area > max_area:
                        max_area = area
                else:
                    width = 0

    return max_area


matrix = [
    ['1', '0', '1', '0', '0'],
    ['1', '0', '1', '1', '1'],
    ['1', '1', '1', '1', '1'],
    ['1', '0', '0', '1', '0']
]

print("Maximum Rectangle Area =", maximalRectangle(matrix))
```

```
Maximum Rectangle Area = 6
```

explaiantion: * Read the matrix. * Check all possible rows. * Find the columns having only 1's. * Count width and height of rectangle. * Calculate area using width × height. * Compare with previous maximum area and store it. * Repeat for all rows and print the maximum area.

3. Scarmble string

We can scramble a string s to get a string t using the following algorithm:

If the length of the string is 1, stop. If the length of the string is > 1, do the following: Split the string into two non-empty substrings at a random index, i.e., if the string is s, divide it to x and y where s = x + y. Randomly decide to swap the two substrings or to keep them in the same order. i.e., after this step, s may become s = x + y or s = y + x. Apply step 1 recursively on each of the two substrings x and y. Given two strings s1 and s2 of the same length, return true if s2 is a scrambled string of s1, otherwise, return false.

In [11]:
```
def isScramble(s1, s2):
    if s1 == s2:
```

```
        return True
    if len(s1) != len(s2):
        return False
    count = {}
    for ch in s1:
        if ch in count:
            count[ch] += 1
        else:
            count[ch] = 1
    for ch in s2:
        if ch not in count:
            return False
        count[ch] -= 1
    for value in count.values():
        if value != 0:
            return False
    n = len(s1)
    for i in range(1, n):
        if isScramble(s1[:i], s2[:i]) and isScramble(s1[i:], s2[i:]):
            return True
        if isScramble(s1[:i], s2[n-i:]) and isScramble(s1[i:], s2[:n-i]):
            return True

    return False




s1 = input("Enter first string: ")
s2 = input("Enter second string: ")
if isScramble(s1, s2):
    print("True")
else:
    print("False")
```

```
True
```

explaination

* here we have used user defined function to check the jumbled stings are the same without sorting it

* by creating and empty dictionary where we can store all the charater and the count of it

* to check we will be -1 of the value if all the dictonary value return zero then its true

*we can also do the same thing with the help of empty list which would have been complicated but easy to understand

4.BUY AND SELL STOCK

You are given an integer array prices where prices[i] is the price of a given stock on the ith day.

On each day, you may decide to buy and/or sell the stock. You can only hold at most one share of the stock at any time. However, you can sell and buy the stock multiple times on the same day, ensuring you never hold more than one share of the stock.

Find and return the maximum profit you can achieve.

In [13]:
```
prices = list(map(int, input("Enter stock prices separated by space: ").split()))
profit = 0

for i in range(1, len(prices)):

    if prices[i] > prices[i - 1]:
        profit += prices[i] - prices[i - 1]

print("Maximum Profit =", profit)
```

```
Maximum Profit = 10
```

EXPLAINATION

* buy the stock when the price is lower than the next day

*sell the stock if it is lesser than the next day

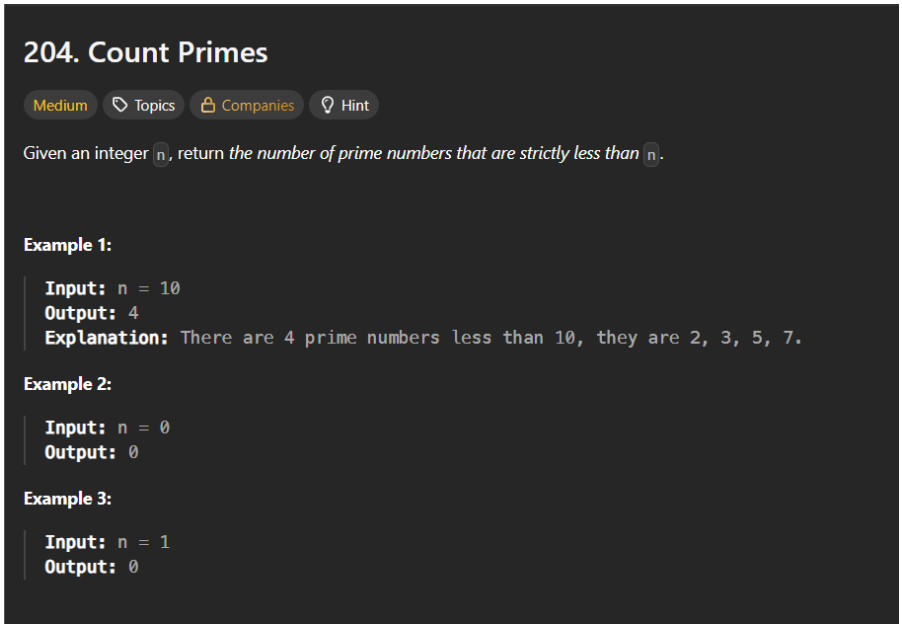
In [ ]:
```

```

SCREENSHOTS OF THE LEETCODE QUE//////GB B GGV

STIONS

1.PRIME NUMBER WITH OUT TRAVESING FOR 0 TO N AND FOR THE SHORTLIST SEGMENT



204. Count Primes

Medium | Topics | Companies | Hint

Given an integer `n`, return *the number of prime numbers that are strictly less than* `n`.

Example 1:

```
Input: n = 10
Output: 4
Explanation: There are 4 prime numbers less than 10, they are 2, 3, 5, 7.
```

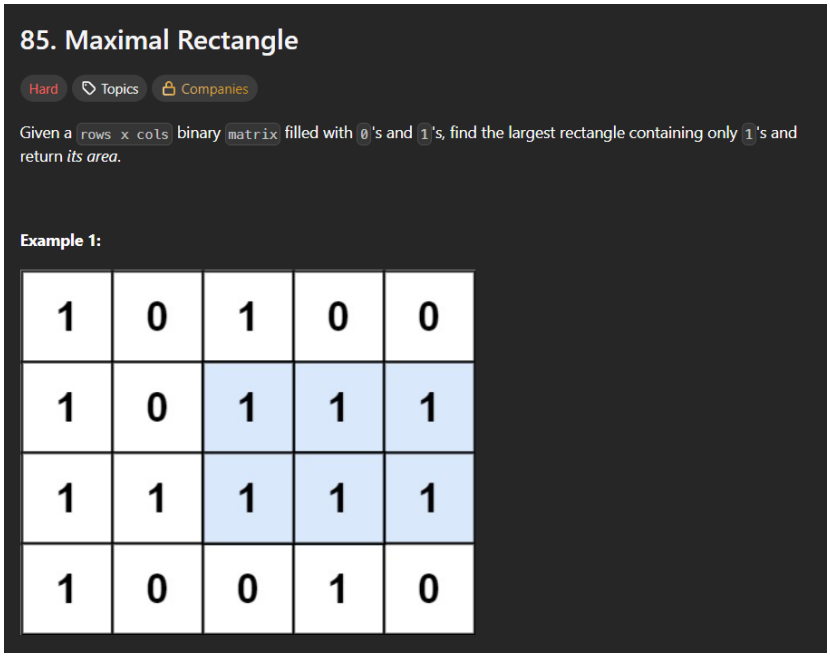
Example 2:

```
Input: n = 0
Output: 0
```

Example 3:

```
Input: n = 1
Output: 0
```

2. Maximal Rectangle



85. Maximal Rectangle

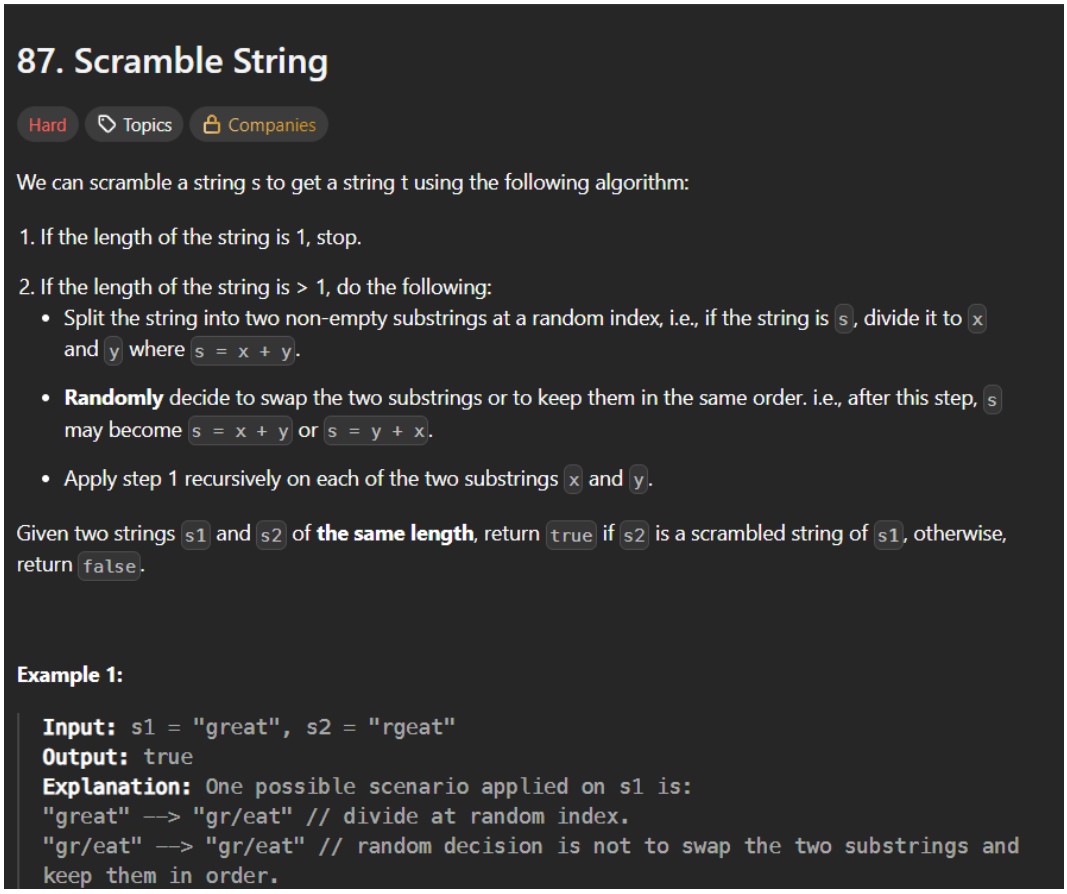
Hard | Topics | Companies

Given a `rows x cols` binary `matrix` filled with `0`'s and `1`'s, find the largest rectangle containing only `1`'s and return *its area*.

Example 1:

1	0	1	0	0
1	0	1	1	1
1	1	1	1	1
1	0	0	1	0

3.SCRAMBLED STRING



87. Scramble String

Hard | Topics | Companies

We can scramble a string s to get a string t using the following algorithm:

1. If the length of the string is 1, stop.
2. If the length of the string is > 1, do the following:
   - Split the string into two non-empty substrings at a random index, i.e., if the string is `s`, divide it to `x` and `y` where `s = x + y`.
   - **Randomly** decide to swap the two substrings or to keep them in the same order. i.e., after this step, `s` may become `s = x + y` or `s = y + x`.
   - Apply step 1 recursively on each of the two substrings `x` and `y`.

Given two strings `s1` and `s2` of **the same length**, return `true` if `s2` is a scrambled string of `s1`, otherwise, return `false`.

Example 1:

```
Input: s1 = "great", s2 = "rgeat"
Output: true
Explanation: One possible scenario applied on s1 is:
"great" --> "gr/eat" // divide at random index.
"gr/eat" --> "gr/eat" // random decision is not to swap the two substrings and
keep them in order.
```

4.SELL AND BUYING STOCK

122. Best Time to Buy and Sell Stock II

Medium | Topics | Companies

You are given an integer array `prices` where `prices[i]` is the price of a given stock on the ith day.

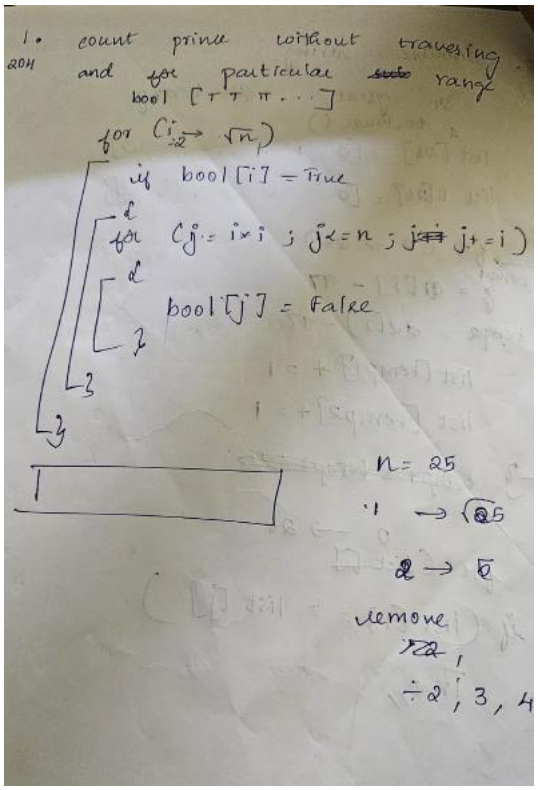
On each day, you may decide to buy and/or sell the stock. You can only hold **at most one** share of the stock at any time. However, you can sell and buy the stock multiple times on the **same day**, ensuring you never hold more than one share of the stock.

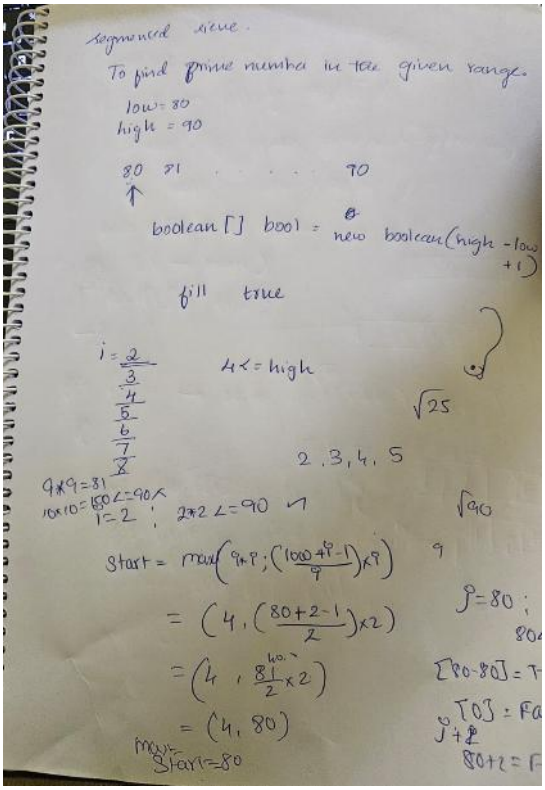
Find and return *the **maximum** profit you can achieve.*

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 7
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4.
Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3.
Total profit is 4 + 3 = 7.
```

BRAINSTORMING SESSION :





algo to check prime

$i = 2 \rightarrow \sqrt{n}$

2 — 25

2 → 5

Remove all divisible of 2, 3, to 5

n = 26

2 → √25

2 → 5

0	1	2	3	4	5	6	7	8	9
T	T	T	T	F	T	F	T	F	F

10	11	12	13	14	15	16	17	18	19	20
F	T	F	F	F	F	F	T	F	T	F

21	22	23	24
F	F	T	F

remove all divide (i × j) → start

2 × 2 = 4

j = j + i

= 6

= 8

= 10

= 12

=

3 × 3 = 9

2. scramble string

S1 = great S2 = tearg

A to lower ()

list [26] = [0 . . . . 0]

list B [26] = [0 . . . . 0]

for (0 → 26)

temp1 = s1[i] - 97

temp2 = s2[i] - 97

list [temp1] + = 1

list [temp2] + = 1

~~temp1 = temp2~~

0 → 26

for (list [i]

if (list [i] = = list [j])

True //

3. largest rectangle

86

get matrix ~~size~~, n x m, 1, 0

~~if (i → n~~

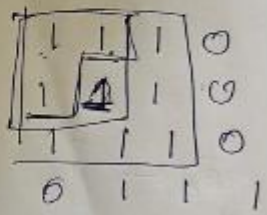
for (i = n)

{

for (j = m)

{

if (i = = 1)



[ ]